Faultline

Find the hidden risk in open-source dependencies before something breaks.

Project pitch report | Dhatri Neerja | 1 November 2026
Status: plan complete, build starting at Stage 0

THE PROBLEM

Stars measure popularity, not health, and the real risk sits where nobody is looking

2016

left-pad

One maintainer unpublished an 11-line npm package, and builds broke across the industry.

2021

Log4Shell

A critical flaw in a logging library that many teams did not know they used, because it arrived transitively.

2024

xz backdoor

An attacker spent years gaining the trust of the overstretched maintainer of a library used across Linux.

> Each incident was a dependency-chain or maintainer-concentration problem. Tools that score one repository at a time cannot see either.

THE NEED

Three groups need an answer that popularity numbers cannot give

Who	Their question	What Faultline gives them
Engineering leaders	Which library should we adopt, and is our current stack still safe?	Ranked recommendation with reasons, plus a stack risk view
Security teams	If package X is abandoned or compromised, what is affected?	Blast-radius query and open advisories
Developer-tool investors	Which projects are rising or quietly declining?	Trend views and early decline flags

> Supply-chain rules such as SBOM expectations and the EU Cyber Resilience Act are pushing teams to know what they depend on, not only what they import.

THE GAP

Existing tools each cover part of the picture, and none we found joins it all up

Existing tool	What it does	What is missing
OpenSSF Scorecard	Checks security practices of one repository	Activity trends, dependency chains, recommendations
deps.dev	Dependency graphs, advisories, Scorecard data	Maintainer-health score, ranked picks, backtest
ecosyste.ms	Indexes packages and dependents	Raw data, not a health judgment
Cauldron, LFX Insights	Hosted community health analytics	Mostly per-project views; no recommendations
Snyk, Aikido	Health signals inside security products	Closed scoring
Stars and downloads	Popularity	Whether maintainers are still there

> The claim is deliberately narrow: no free tool in our survey combines transitive and maintainer risk, a reasoned recommendation and a published backtest.

Preliminary survey. Each tool's current features will be re-checked before launch.

THE IDEA

Faultline tracks health, maps hidden risk, recommends what to use, and tests itself

Track

Is this package healthy?

Eight activity and risk signals become one transparent score, with trends and early decline flags.

Map

Where does risk hide?

A knowledge graph shows transitive risk, maintainer concentration, blast radius and licence conflicts.

Recommend

What should I use?

Describe a need and get a ranked pick, with every reason traced to the score.

Prove

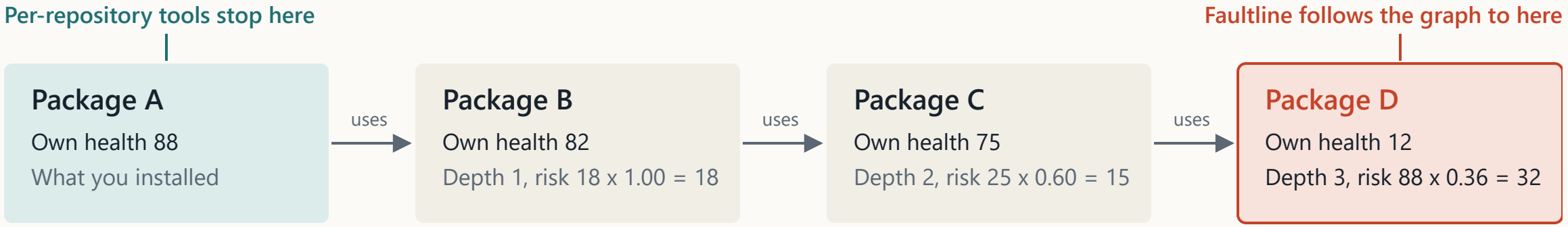
Does the score work?

A published backtest against stars, commit recency and OpenSSF Scorecard.

> Version 1: about 2,000 PyPI packages, refreshed about every 30 minutes, planned at $0 running cost on free tiers.

THE CORE INSIGHT

A package can look healthy on its own page and still rest on a weak link three levels down



Dependency risk for Package A = 32, driven by Package D

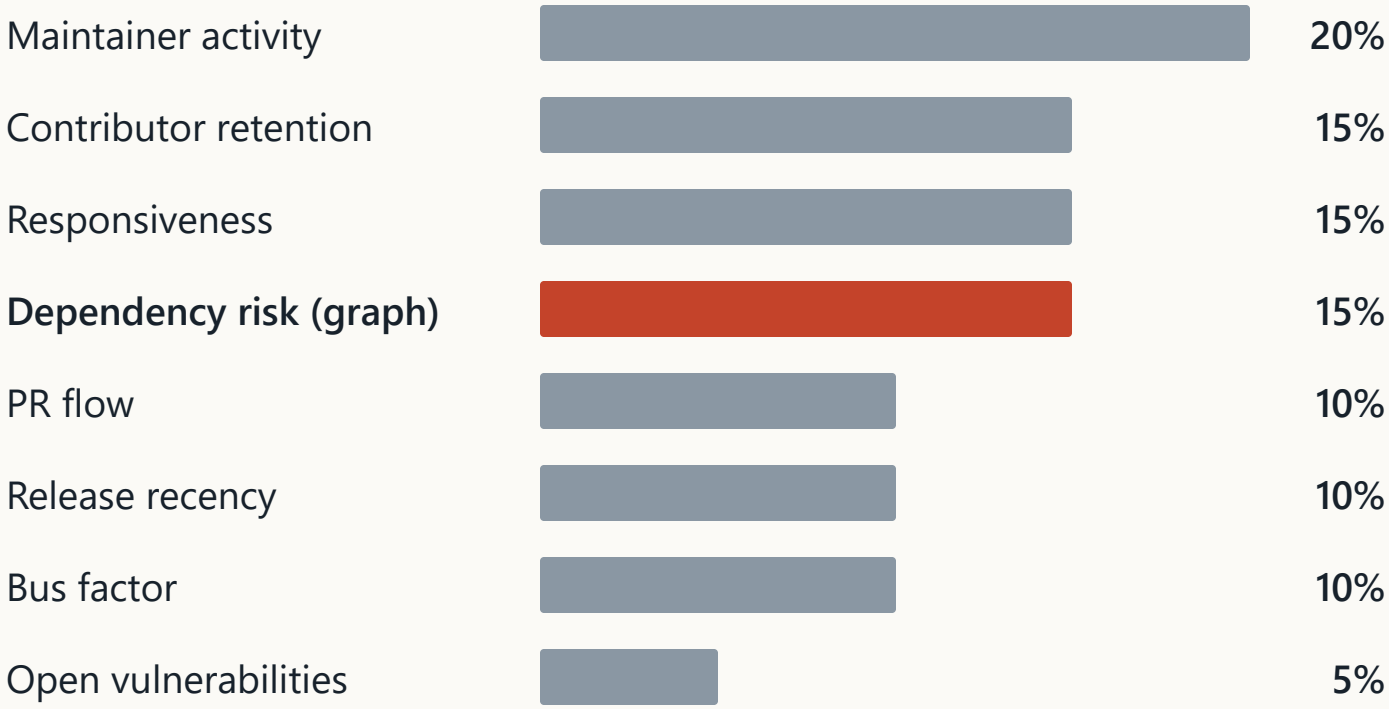
Worst dependency wins, discounted by 0.6 per level: (100 - own health) x 0.6^(depth - 1)

The same graph answers who is affected if D is abandoned (blast radius), and whether one person holds several critical packages.

Illustrative example. Packages and numbers are invented to show the formula; no real package was scored.

THE SCORE

One transparent, versioned formula that anyone can recompute and challenge



Percentile-based. Each signal becomes a 0 to 100 percentile within the tracked set, then combines by weight.

Not circular. Dependency risk uses each dependency's own health, without its dependency term.

Versioned. Every score row stores its formula version, so results can be reproduced.

Disciplined. Weights are starting values, tuned only on the backtest's training window.

Start weights for formula v1; they sum to 100%.

THE RECOMMENDATION

Describe a need, and get a ranked pick with reasons that trace back to the data

Example need: "Python library for parsing PDFs, MIT licence, for a commercial product."

1. Find

Vector search over READMEs, descriptions and topics

2. Filter

Remove the wrong language or licence

3. Rank

Health, adoption, dependency risk, vulnerabilities

4. Explain

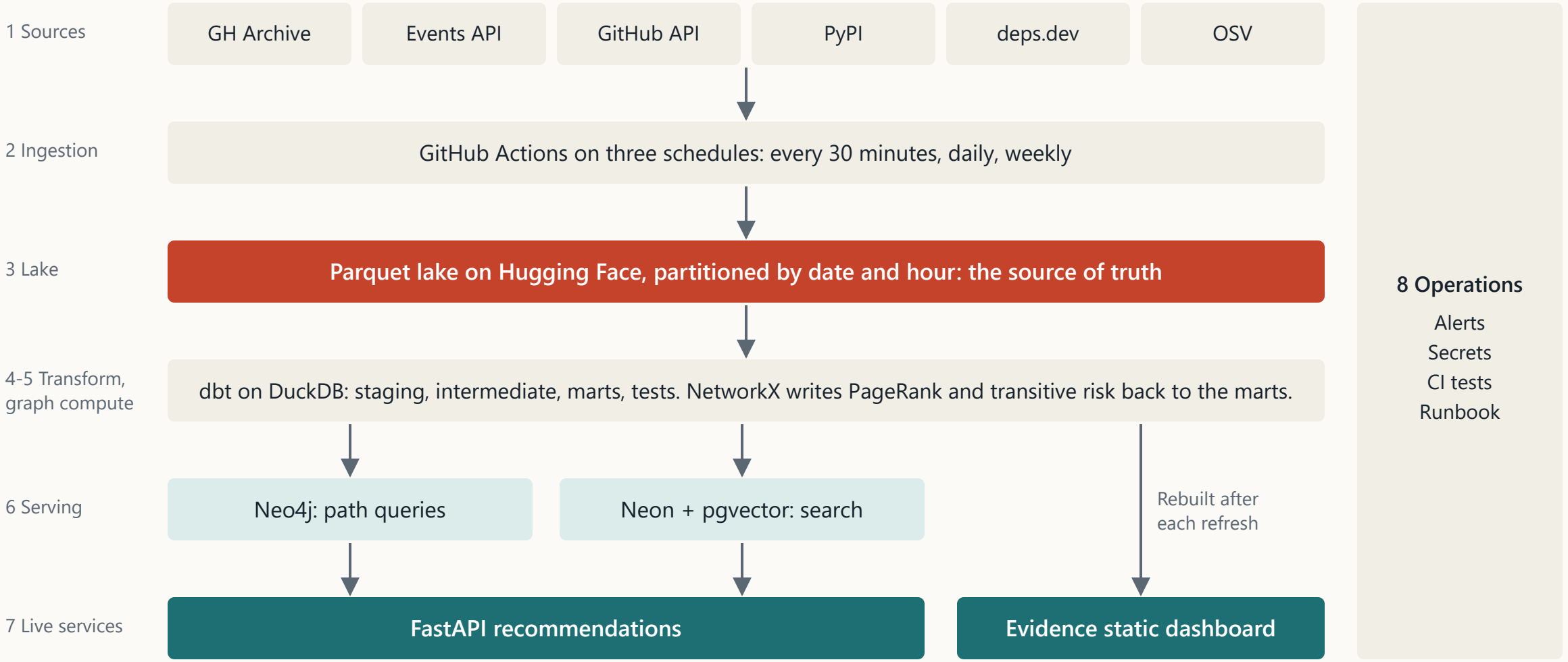
Templates filled from score components

> "Pick A: active maintainers and broad adoption. Avoid B: one maintainer, and it relies on an abandoned package."

Explanations use templates, not a language model, so they are free, testable and cannot invent a fact.

THE ARCHITECTURE

One source of truth, with a purpose-built copy for each live feature



Visitors only read copies, so no page view triggers a heavy query. Any store can be rebuilt from the lake.

DESIGN CHOICES

Every choice is deliberate, free today, and has a named path to larger scale

Layer	Choice	Why	At larger scale
Orchestration	GitHub Actions cron	No server to run	Airflow or Dagster
Processing	DuckDB + dbt	Filtered data fits on one machine	Warehouse or Spark
Lake	Parquet on Hugging Face	Permanent, columnar, versioned	Object storage + Iceberg
Graph	Neo4j AuraDB Free	Readable multi-hop path queries	Paid Neo4j or Memgraph
Search	pgvector on Neon	Vectors and filters in one query	Dedicated vector database
Serving	FastAPI, Evidence	Typed API; static site survives failures	Cloud Run, Superset

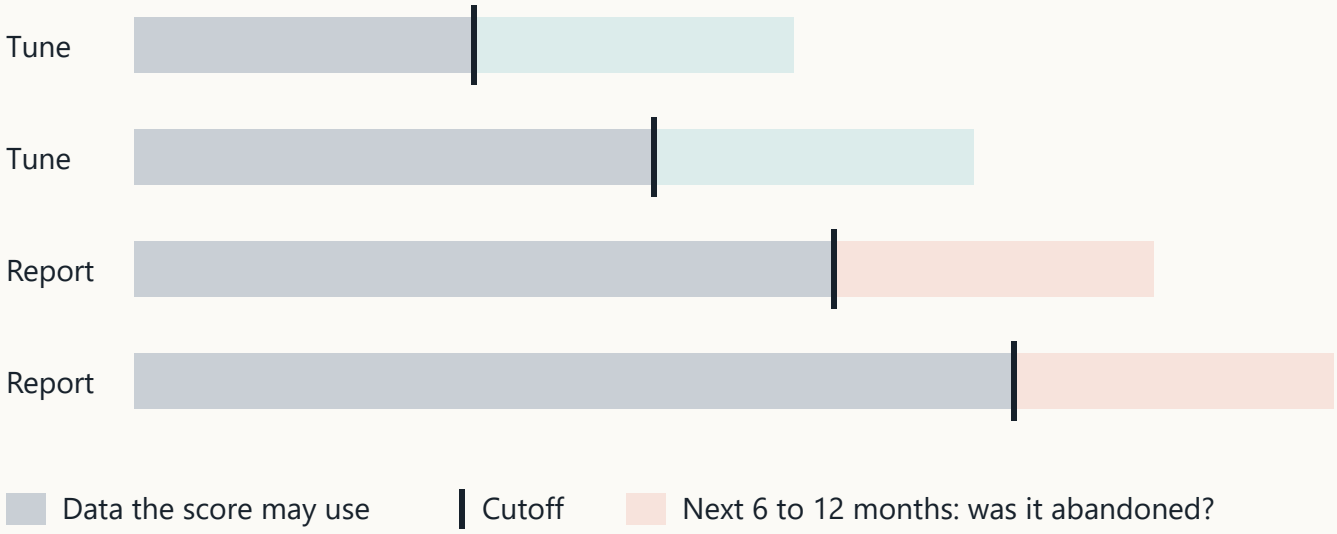
> **Why batch, not Kafka?** Package health moves over days and weeks, so a 30-minute refresh supports the same decisions with no always-on server.

> **Safe re-runs.** The daily GH Archive run overwrites the 30-minute numbers by date and hour, so data is corrected and never double-counted.

THE PROOF

The score has to earn trust: it will be tested on history, against simple baselines

Rolling cutoffs (illustrative spacing)



Later data never leaks into the score. Package lists are rebuilt as of each cutoff.

Compared against

Stars alone, last-commit recency alone, OpenSSF Scorecard

Reported

Precision and recall, precision at the bottom 5% of scores, PR-AUC, and lead time in months

Graph test

The score with and without the dependency term

Pre-committed

"Abandoned" is defined and committed before any data is examined

No results yet. The hypothesis is that the combined score beats each baseline; Stage 6 will confirm or reject it, and misses will be published.

THE ROADMAP

Eight gated stages in about 13 weeks, with a live MVP around week 6

Stage	Week 0 to 13 (upper estimate)	Gate to pass
0 Setup and definitions		Definitions committed before data is examined; limits confirmed
1 Daily pipeline		7 straight days loaded; re-runs give identical row counts
2 Models and score v1		All dbt tests pass; 95%+ of mapped packages scored
3 Dashboard and alerts (MVP)		Public URL; a broken run alerts and the site keeps last good data
4 Knowledge graph		Risk queries work for every package; under 80% of the graph cap
5 Search, recommendation		20 test queries give sensible top 3s, with traceable reasons
6 Backfill and backtest		Results published for all baselines, with and without the graph
7 Launch and write-up		A newcomer understands the project from the README in 5 minutes

Grey: foundations. Teal: MVP. Red: the differentiators. Assumes 10 to 15 hours a week; estimates total 11 to 13 weeks.

DEMO POINTS

The planned demo tells one story, with each moment unlocked by a stage gate

#	Show	The point it makes	Ready after
1	Dashboard: one package's trend	Health is measured from activity, not stars	Stage 3
2	Last-updated time and formula version	Data is fresh, and the score is reproducible	Stage 3
3	Break a run live, or show a recorded failure	An alert fires, and the site keeps the last good data	Stage 3
4	Graph: the three-levels-down query	A healthy-looking package has a hidden weak link	Stage 4
5	Blast radius for one critical package	What is affected if it is abandoned or compromised	Stage 4
6	Type the PDF-library need into the API	A ranked pick, with reasons traced to the score	Stage 5
7	Backtest results table	How the score did against the baselines, misses included	Stage 6

> Today's presentation walks through the design. The live demo follows as each stage ships.

RISKS AND RESPONSIBILITY

The main risks are known, and each one already has a mitigation in the plan

Risk	Mitigation
GitHub rate limits: about 4,000 calls an hour against a 5,000 limit	ETags, GraphQL batching, or a 30-minute refresh for only the most active repositories
Incomplete PyPI-to-GitHub mapping	PyPI and deps.dev links, a manual override file, and a public "unmapped" report
Neo4j free-tier cap or idle pause	Stay under 80% of the cap; rebuild from the lake; Postgres as fallback
Late or disabled scheduled jobs	Show last-updated time and alert on staleness
A wrong or gamed score	Open components, an "insufficient data" state, published misses
Scope creep	Ship the MVP, then the graph, before anything new

> **Maintainers are people.** Concentration is shown as a property of a package, never a ranking of people, and public wording is "needs support", not "dying".

SUMMARY

Faultline turns a popularity contest into evidence about what is safe to depend on

~2,000
PyPI packages in version 1

~30 min
planned refresh interval

$0
planned running cost on free tiers

What happens next: Stage 0

1. Lock the "abandoned" definition and metric rules before examining any data.
2. Choose the 2,000 packages and map them to GitHub repositories.
3. Confirm every free-tier limit, and create the accounts.

SPEAKER NOTES

What to say on each slide (about 12 minutes in total)

1. Title (20 sec)
Say the name, then the one-line pitch. State up front that this is a plan and the build starts now.

2. The problem (1 min)
Open with: "How do you decide a library is safe? Most of us check the stars." Walk through the three incidents quickly. The point is the pattern, not the details.

3. The need (40 sec)
Pick the group closest to your audience and speak to it directly. Mention regulation only as a tailwind.

4. The gap (1 min)
Be fair to these tools, since they are good at what they do. Say "in our survey" when making the claim, and note the features will be re-checked.

5. The idea (40 sec)
Four verbs: track, map, recommend, prove. Stress that the fourth is the unusual one.

6. Core insight (1 min 15 sec)
This is the key slide, so slow down. Follow the chain left to right, land on Package D, and explain why risk 32 comes from three levels down. Make clear the example is invented.

7. The score (45 sec)
Do not read the bars. Point to the red one, say it is what others lack, then give the "versioned" line.

8. Recommendation (45 sec)
Read the example need aloud. Explain that templates were chosen over an LLM because an explanation must not invent a reason.

9. Architecture (1 min 15 sec)
Read it top down. Three ideas: one lake as the truth, one copy per access pattern, and visitors only read copies.

10. Design choices (45 sec)
Show the thinking, not the logos. Pick one row, then the Kafka answer.

11. The proof (1 min 15 sec)
Say plainly that there are no results yet. Explain "tune early, report late", and why the definition is committed before looking at data.

12. Roadmap (45 sec)
Gates matter more than dates. Each stage has a pass test. Point out that the MVP comes first, then the differentiators.

13. Demo points (45 sec)
Describe the story you will show. Do not imply any of it runs today.

14. Risks (45 sec)
Name the rate-limit risk first, because it is the most concrete. Close on the maintainers line, since it shows judgment.

15. Summary (30 sec)
Repeat the opening line about stars, give the three numbers as plans, and name Stage 0 as the next step.

SPEAKER NOTES

Likely questions and short answers

Question	Answer
Why three stores for 2,000 packages?	Access patterns need it, not scale. The cost is more moving parts, contained because the other stores are rebuildable copies of the lake.
Why not stream with Kafka?	Health moves over days and weeks. A 30-minute batch supports the same decisions with no always-on server.
How do you handle late or duplicate data?	Writes replace a date-hour partition, and the daily run overwrites the 30-minute numbers. A retry replaces and never appends.
How do you know the data is right?	dbt tests on keys, nulls, values, row counts and freshness. A failed test stops the publish step.
How do you map a package to its repo?	PyPI project URLs and deps.dev links, plus a manual override file. Unmapped packages are reported, not dropped.
Why DuckDB and not Spark?	After early filtering the data fits on one machine, and the dbt SQL can move to a warehouse later.
How does this scale?	Swap Actions for Airflow, the dataset for Iceberg, DuckDB for a warehouse, and Neo4j Free for paid. The layers stay the same.
Why templates and not an LLM?	They are deterministic, testable and free, and every sentence traces to a score component.
What if the API sleeps or Neo4j pauses?	The static dashboard is unaffected, the API cold-starts, and pipeline writes keep Neo4j active. The graph can be rebuilt from the lake.
What if the backtest fails?	Then we publish that. A clear negative result is still useful, and the graph comparison shows what to change.